

Malware Analysis - Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Claude to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

CHAPTER 1: Introduction

Malware Analysis and Incident Forensics

The Anatomy of an Intrusion: The Cyber Kill Chain

Every cyberattack, however sophisticated or crude, can be decomposed into a sequence of discrete stages. This decomposition is not merely academic: it gives defenders a shared vocabulary for describing where in an attack's lifecycle a particular piece of evidence belongs, and it gives malware analysts a checklist of behaviors to expect and hunt for when dissecting a sample. The most widely adopted model for this decomposition is the **intrusion kill chain**, originally formulated by Lockheed Martin. It describes, stage by stage, the process by which perpetrators carry out a cyberattack, from the earliest scouting activities through to the final exploitation of the compromised environment.

The chain begins with **Reconnaissance**, the stage in which the attacker studies the target in search of vulnerabilities. This search is not limited to software flaws: it extends to the identification of weak points in an organization's technical stack (outdated software, misconfigured services) as well as its human element (employees who might be susceptible to social engineering). Reconnaissance is the intelligence-gathering phase that determines what avenues of attack will even be feasible.

Once a vulnerability or weakness has been identified, the attacker moves to **Weaponization**. This is the stage in which the attacker crafts an artifact capable

of exploiting the discovered vulnerability — for instance, engineering a payload that triggers a buffer overflow in a specific piece of software. Weaponization transforms abstract knowledge of a flaw into a concrete, usable exploit.

The third stage is **Delivery**, the mechanism by which the weaponized artifact is transported to the victim. Delivery can take many forms: a phishing email carrying a malicious attachment, a booby-trapped USB drive left in a parking lot, a drive-by download from a compromised website, and so forth. The delivery mechanism is often what a defender first observes, since it is the point at which the malicious content crosses into the victim's environment.

Exploitation follows delivery: this is the moment at which the vulnerability identified during reconnaissance and targeted during weaponization is actually triggered, granting the attacker some initial foothold or code execution capability on the target system.

After exploitation succeeds, the attacker typically seeks to secure and extend that foothold. This is where the behavioral properties that malware analysts are trained to recognize come into play.

Persistence

Persistence is the property by which malware achieves and guarantees its own survivability within the compromised system over time. Consider a straightforward but illustrative scenario: the infected machine is rebooted, meaning the computer is fully shut down and then powered back on. If persistence has been correctly achieved, the malware will resume execution automatically the moment the system boots again, without any further action required from the attacker. In other words, persistence is about ensuring the malicious code survives beyond the singular moment of infection.

To achieve this survivability, malware typically must tamper with the operating system's configuration — modifying registry keys, scheduled tasks, service definitions, startup folders, library search paths, or similar mechanisms that the OS consults when deciding what code to execute automatically. This tampering process can vary enormously in character: it might be conducted in a very noisy, conspicuous fashion, or it might be executed with extreme care to avoid detection. Regardless of the specific technique, the changes made to achieve persistence are frequently designed to be hidden from casual inspection, since an administrator who stumbles upon an unfamiliar autorun entry might well investigate and remove it.

Uniqueness ("Single-Instance" Behavior)

A second behavioral property commonly exhibited by malware is **uniqueness**, sometimes described as "single-instance" execution. Malware authors generally want to guarantee that only one copy of their malicious code is running on a given machine at any time. This might seem counterintuitive at first glance — surely more running instances of malicious code would only benefit the attacker? — but in practice, multiple concurrent executions of the same malware are frequently meaningless or even actively harmful to the malware's own operation. Two instances might compete for the same system resources, interfere with each other's tasks, or produce undesired side effects. A vivid illustration of this danger is ransomware: if two instances of the same ransomware strain began encrypting the filesystem simultaneously, the result could be file corruption or an encryption process that breaks its own functionality, rendering the ransom note's promise of a decryption key useless even to the attacker. To prevent this outcome, malware commonly performs start-up checks — for instance, creating a named mutex or checking for a marker file — to verify whether the machine has already been infected. If an existing infection is detected, the new instance will typically decline to create a second infection, though it might instead choose to update the existing one.

Targeting

Not all malware is indiscriminate. Some threats are explicitly designed to infect only specific, pre-selected victims, and such malware needs a way to recognize whether it has landed on the intended target or on some other, irrelevant machine. This behavioral property is called **Targeting**. When a sample lands on a machine that is not the intended target, it might choose to terminate its own execution quietly, or it might instead disguise itself by behaving like an entirely different, more innocuous piece of malware or software, thereby throwing off analysts who happen to be observing it.

The mechanisms available to malware for recognizing the "right" victim machine are numerous. A classic technique, especially prevalent among banking trojans, is checking regional and language settings — a piece of malware targeting Eastern European banks, for example, might terminate immediately if it detects that the system locale is set to English (United States). Another approach is checking hardware and software characteristics of the machine, such as the presence of a specific CPU model, particular installed software, or particular configuration settings. Malware can also check for the presence of specific external devices, or

search for known documents that would only be present on a machine belonging to the intended target organization. In essence, the malware fingerprints the machine it has landed on in order to determine whether it matches the profile of the intended victim.

Obfuscation and Evasion

Eventually, defenders will notice something is wrong, and analysts will begin dissecting the malware to understand how it operates. Malware authors know this is inevitable and therefore invest considerable effort into making their creations difficult to analyze. This concern gives rise to two closely related properties:

Obfuscation and Evasion.

Obfuscation refers to techniques that make data and code structures genuinely difficult to interpret, even when an analyst has full access to the binary. This can involve scrambling the malware's own code and data structures using complex, deliberately convoluted schemes, so that any easily discoverable hints of the sample's malicious nature are hidden from casual static inspection. Malware may also inject its code into the address space of other legitimate processes in order to conceal its real identity from process-monitoring tools, or it may employ a broader arsenal of techniques specifically engineered to hamper the analyst's ability to correctly understand its behavior. Malicious code might also be spread across multiple components or delivery stages, making it harder for an analyst to assemble the complete picture of how the attack functions, since no single artifact contains the whole story.

Evasion, meanwhile, refers to the malware's ability to actively hide its own presence and effects while operating, and to present a different, innocuous appearance if it detects that it is being analyzed — for instance, by a sandbox, a debugger, or antivirus software. The overarching goal of both obfuscation and evasion is to allow the malware to remain undercover on the infected machine for as long as possible, maximizing the window during which it can operate undetected. There are, however, notable exceptions to this "stay hidden" philosophy: ransomware, for example, has no interest whatsoever in remaining covert once it has finished encrypting the victim's files, since its entire business model depends on making its presence loudly and unmistakably known in order to extract a ransom payment.

Fingerprinting and Beaconing

Once the attack has succeeded in compromising the machine, the malicious software typically needs to inform the attacker that the operation was successful. This is accomplished by reusing many of the same techniques employed for targeting: the malware inspects the infected system and collects descriptive information about it, then transmits this description back to the attacker, commonly via a remote server under the attacker's control. This notification stage is referred to as **Beaconing** — the malware "beacons" its presence and characteristics back to its controller, confirming that the infection has taken hold and providing the attacker with the details needed to manage the newly compromised asset.

Communication

Beyond the initial beacon, malware frequently maintains ongoing **Communication** with external Command and Control (C2) infrastructure. This communication channel serves multiple purposes: it allows the malware to receive further instructions and commands from the attacker, to update its own internal components (effectively self-updating in the field), and to exfiltrate data harvested from the victim machine back to the attacker.

The methodologies used to implement this communication are impressively heterogeneous, reflecting the attacker's need to blend malicious traffic in with the ordinary noise of network activity. Communication may take place through direct links to attacker-controlled servers, but it can equally be routed through online storage services, from which the malware simply reads commands as though it were retrieving an ordinary file — from a defender's vantage point, all that is visible is a request to a legitimate cloud storage provider, with nothing to obviously flag it as malicious. Social media platforms present another channel: fake accounts are sometimes used to host commands or facilitate data exfiltration, again blending in with ordinary internet traffic. Malware may also communicate through network services, most notably DNS queries, using the domain name resolution system as a covert channel for encoding and transmitting information. Whatever channel is chosen, the data exchanged over it is frequently obfuscated, ensuring that even if the traffic itself is intercepted, its contents remain opaque to an observer.

CHAPTER 2: x86 Assembly

The Nature of Binary Analysis

The vast majority of the malware samples an analyst will encounter arrive in the form of executables — binary content that a processor can directly execute. Fortunately, an analyst does not need to painstakingly examine every single byte of a malware sample; only specific portions of a binary typically warrant the deep, instruction-by-instruction scrutiny that reverse engineering demands, and there exists a range of tools designed to help focus that effort where it matters most.

At the most fundamental level, a computer executes a sequence of instructions. Malware, however, often "writes" these instructions in ways that diverge meaningfully from how a standard, benign program would be structured. A malicious sample might employ specific instructions that accomplish a given task in a highly inefficient or unusual way, purely because that unusual approach helps it evade signature-based detection or confuses automated analysis tools.

Disassemblers allow an analyst to examine code starting from the executable's declared entry point — the address, specified within the executable file's header, that dictates which instruction must be executed first when the program is loaded and run. A critical subtlety of the x86 architecture is that instructions are encoded as sequences of bytes, and the interpretation of any given byte depends entirely on the context established by what has already been read and decoded up to that point. Ordinarily, when reading a book, one begins at the first sentence and proceeds linearly to the end. Compiler-generated code behaves in a broadly analogous fashion: control flow, while it does branch and jump, tends to respect clean boundaries between instructions. Malware, by contrast, might begin reading its own instruction stream in an unusual order, jump abruptly to a location that is not the "obvious" next instruction, and then continue reading from what amounts to the middle of what a naive disassembler would have interpreted as a completely different instruction. The effect is somewhat like a reader whose eyes jump mid-sentence to a different point on the page and continue reading from there, deriving a sequence of words that bears no resemblance to what the original, linear reading would have produced. This deliberately anomalous structuring does not arise by accident — it exists specifically because the malware's author does not want the code to be easily analyzed. Malware may go even further and rewrite its own code at runtime, meaning that the bytes present in the file on disk do not even fully represent what will actually execute in memory. Because of these evasive properties, a careful analyst must, at least in principle, be prepared to analyze code one instruction at a time in order to achieve precision, rather than relying on the assumption that instructions will be laid out consecutively and unambiguously.

A useful class of tools that alleviates some of this burden is the family of **decompilers**, such as Ghidra and Hex-Rays, which attempt to reconstruct a high-level, C-like representation of a fragment of binary code. There are certain instruction sequences in assembly that are extremely verbose when read in their raw form but that correspond to easily recognizable, common patterns; decompilers are designed to recognize these patterns and collapse them into more legible high-level constructs. For an analyst working through their first malware sample, however, even with these tools, the process can consume a great deal of time, and it is worth being candid about their limitations: decompilation is a process far from perfect. Information is irretrievably lost during the original compilation step — variable names, precise data types, and other identifiers that a human programmer relied upon simply do not survive translation into machine code, so a decompiler can only ever offer its best guess at reconstructing plausible substitutes. Furthermore, if obfuscation has been deliberately applied to the binary, decompiling the resulting obfuscated code will simply produce an obfuscated, and correspondingly unhelpful, reconstructed source — the decompiler cannot see through obfuscation any more effectively than a human reading raw assembly can. Malware authors are also known to specifically employ anti-analysis techniques engineered to confuse or break decompilers outright. Despite all these caveats, decompilation remains, on balance, very helpful for reverse-engineering professionals, since even an imperfect, obfuscated-looking C-like reconstruction is often faster to reason about than raw assembly.

It is worth stating plainly, as a foundational fact underpinning this entire subject: malicious code is not equivalent to compiler-generated code. Where a compiler produces code following predictable, well-understood idioms optimized for correctness and performance, a human or semi-automated malware-crafting process introduces obfuscation of control and data flows, subtle semantics such as an instruction that deliberately jumps into the middle of another instruction's byte sequence (exploiting exactly the byte-stream ambiguity described above), hand-written instruction sequences such as custom API-hashing algorithms used to resolve system calls without leaving human-readable strings in the binary, self-modifying code, and a range of other decoys designed specifically to mislead automated and manual analysis alike.

CPU and Memory Basics

To reverse engineer code effectively, an analyst must understand the substrate on which that code executes: the central processing unit and its relationship to

memory.

Programmers, when writing high-level source code, think in terms of an essentially unlimited number of variables, each of whatever type is convenient for the problem at hand. The central processing unit, however, offers no such luxury. From the CPU's perspective, it is simply a collection of memory cells of specific, fixed sizes; there exist some cells of larger sizes as well, primarily for optimization purposes in domains like multimedia processing. Within the CPU itself, these cells are known as **registers**, and here the gap between the programmer's abstraction and the hardware's reality becomes especially stark: rather than an unlimited supply of named variables, the programmer working at this level has access to only a small, fixed number of general-purpose registers. On a modern (64-bit) CPU, there are additional special registers that are not directly controlled by the programmer — some exist purely to support debugging, and others are reserved for use by the operating system.

The registers directly usable by ordinary programs are known as the **general-purpose registers** (commonly labeled EAX, EBX, ECX, EDX, and so on in the 32-bit x86 world). The key architectural fact to internalize here is the tension between abstraction and hardware reality: high-level programming languages give the illusion of unlimited storage locations, while the CPU physically provides only a handful of extremely fast storage cells. Because registers offer dramatically faster access than main memory, any piece of data that will be accessed frequently benefits enormously from being kept in a register rather than being repeatedly fetched from and written back to memory.

This tension between what a programmer or compiler wants to express and what the hardware can actually provide is formalized through a contractual abstraction layer called the **Instruction Set Architecture (ISA)**. The ISA is an abstract model that describes precisely what a programmer needs to know in order to program the machine: for x86, this includes the fact that there are eight general-purpose registers, the formats in which arithmetic instructions are expressed, and so on. The ISA functions as an interface, or contract, between software and hardware: humans (or, far more commonly, compilers acting on their behalf) produce ISA-conformant assembly code, and this assembly code is then translated by an assembler into a binary format that the physical hardware can directly consume and execute. Intel originally defined the x86 ISA for its 8086 processor, developed between 1976 and 1978. Over the following decades, later extensions to the architecture have consistently preserved backward compatibility with this original design — a design decision with real consequences for malware analysis,

since even today, most malware samples encountered in the wild are still compiled as 32-bit binaries rather than 64-bit. The x86 ISA is also known by the alternate designation IA-32, and its 64-bit successor is referred to as x86-64, or x64 for short.

The prevalence of 32-bit malware, despite essentially all modern machines running 64-bit CPUs, deserves a moment's reflection. All contemporary machines are built around 64-bit CPUs, but these CPUs remain capable of executing 32-bit software, a capability that exists specifically to preserve backward compatibility with older programs. Windows, in particular, goes to considerable lengths to support the execution of older 32-bit software. This continued support for an older execution mode is convenient for a malware author for a very specific reason: it creates additional gaps and inconsistencies between how 32-bit and 64-bit code is monitored, sandboxed, and defended, gaps that a piece of malware can exploit to evade modern defenses. In addition, there remains a nontrivial fraction of machines in the world — particularly older or specialized systems — that are only capable of running 32-bit software at all, so 32-bit malware retains a broader potential victim pool. For both reasons, it remains convenient, from an attacker's perspective, to write malware that targets the 32-bit execution environment.

Word Size and Endianness

A **word** is understood to be the natural data unit for a given CPU design — the size of data that the processor's arithmetic and logic circuitry is fundamentally built to handle in a single operation. For x86 processors specifically, the native word size is 32 bits. This was not always the case: the original word size for the x86 lineage was 16 bits, and it is precisely for this historical reason, rooted in backward compatibility, that x86 instructions still assume, by default, that a "word" operand is 16 bits long, while a 32-bit operand is instead referred to using the distinct term "dword" (double word). The common IA-32 data type nomenclature, then, comprises byte (8 bits), word (16 bits), and dword (32 bits).

Memory itself, regardless of how large a logical data unit a CPU register might hold, always operates at the byte level — every individually addressable location in memory holds exactly one byte. This raises an immediate practical question: if a register can hold a 32-bit (4-byte) quantity with no difficulty, how does one write that same 32-bit quantity out to memory, which can only address and store data one byte at a time? The answer requires a rule that determines, precisely, in what order the individual bytes composing a multi-byte value should be written to (and

subsequently read from) consecutive memory addresses. This rule is known as **Endianness**.

Consider a concrete example: suppose the goal is to store the dword value 0xABADCAFE at memory address 0x1000. This 32-bit value decomposes into four constituent bytes: AB (the most significant byte, MSB) and FE (the least significant byte, LSB), with AD and CA in between. There are exactly two conventions for how to lay these four bytes out across the four consecutive memory addresses 0x1000 through 0x1003. Under **Big Endian** encoding, the most significant byte is written first, so address 0x1000 would hold AB, followed by AD, CA, and finally FE at address 0x1003. Under **Little Endian** encoding, the convention is reversed: the least significant byte is written first, so address 0x1000 would hold FE, followed by CA, AD, and finally AB at address 0x1003. The x86 architecture uses Little Endian encoding throughout. This detail is far from a mere curiosity for the malware analyst: any time raw memory or a hex dump is being read and interpreted as a multi-byte numeric value or pointer, the bytes must be mentally (or programmatically) reversed to reconstruct the intended value.

Registers in Detail

As already noted, there exists a fundamental mismatch between the programmer's mental model, in which arbitrarily many named variables can be created, and the CPU's physical reality, in which computation is confined to a small number of registers. Registers hold data with extremely fast access time compared to main memory, which is precisely why they are used as the CPU's primary working storage during computation.

Among the registers available on an x86 CPU, several categories are worth distinguishing. The **general-purpose registers** (GPRs) can store either ordinary data values or memory addresses (pointers) — the CPU imposes no rigid distinction between the two; it is purely a matter of how the surrounding code chooses to interpret the bits held within a given register. The **status register** holds a collection of truth values (individual bits, each with a specific meaning) reflecting the state of the processor immediately after the execution of the most recent instruction — this is heavily used, for instance, in evaluating conditional branches. The **program counter** holds the memory address of the instruction currently being executed, effectively tracking the CPU's position within the running program. Beyond these, there exist many other, more specialized registers serving various architectural purposes. On the x86 architecture specifically, the general-purpose registers are 32 bits wide.

Accessing Registers

The x86 architecture provides eight general-purpose registers: commonly referred to by the letters A, B, C, D, along with SI and DI (source index and destination index), plus two registers with special conventional roles. The **ESP** register conventionally points to the top of the stack, and the **EBP** register is often set equal to ESP at the beginning of a function's execution, establishing what is called a base pointer for that function's local data — though, when a function chooses not to use this convention, EBP remains available for general-purpose use like any other register.

The prefix **E** on a register name (as in EAX, EBX, and so on) indicates that the full 32-bit size of the register is being referenced; this E prefix stands for "Extended." When the E prefix is absent, only the least significant 16 bits (that is, the lowest 2 bytes) of the register are manipulated — so, for instance, AX refers to the lower 16 bits of EAX. Furthermore, for the four registers A, B, C, and D specifically, the two least significant bytes of that 16-bit portion can themselves be addressed individually: the high byte of that 16-bit segment is accessed using a name like AH ("A high"), and the low byte is accessed using a name like AL ("A low"). This layered addressing scheme — a full 32-bit register containing a 16-bit sub-register, which in turn contains two individually addressable 8-bit sub-registers — reflects the architecture's historical evolution from 8-bit, to 16-bit, to 32-bit generations, with each new generation extending rather than replacing what came before.

Two particularly important registers cannot be accessed directly by ordinary instructions: the program counter, referred to on x86 as **EIP**, and the status register, referred to as **EFLAGS**. Both are updated implicitly as a side effect of instruction execution rather than being directly writable the way a general-purpose register is.

The Instruction Cycle

The CPU operates according to the classic **fetch-decode-execute cycle**. During the **fetch** stage, the CPU reads the instruction located at the memory address currently held in EIP. During the **decode** stage, the control unit interprets the raw bytes just fetched to determine precisely what operation and operands the instruction specifies. During the **execute** stage, the specified computation is actually carried out, whether that involves the arithmetic logic unit (ALU) performing some calculation or simply moving data between locations. After

execution completes, EIP ordinarily advances to point at the next instruction immediately adjacent in memory — unless the instruction that was just executed explicitly altered the control flow, for instance via a jump, in which case EIP is instead set to whatever destination address the jump specified.

Instructions on x86 vary considerably in their encoded size, ranging anywhere from 1 to 15 bytes, and are stored consecutively in memory along with any immediate operands — meaning literal data values or addresses — that they require as part of their operation. Register operands, by contrast, are not stored as separate bytes following the instruction; instead, they are embedded directly within the binary representation of the instruction's opcode itself. This is illustrated by the following pair of encodings:

```
b8 01 00 00 00      mov eax, 0x1
bf 01 00 00 00      mov edi, 0x1
```

Here, both instructions move the same immediate value (0x1, encoded as the four bytes 01 00 00 00 in little-endian order) into a register, but the specific destination register — EAX in the first case, EDI in the second — is encoded not as a separate operand byte sequence but as part of the opcode byte itself (b8 versus bf). This is a direct illustration of the earlier point that register operands live inside the opcode's own binary representation.

Memory Map of a Process

To fully appreciate how a running program is laid out, it is useful to consider a simplified view of the virtual address space of a 32-bit Windows program. In this simplified model, the lower 2 gigabytes of the address space are reserved for user-space code and data, while the upper 2 gigabytes are reserved for kernel-mode code, inaccessible to ordinary user-mode processes.

Within the user-space portion of the address space, several distinct regions can be identified. At the very base of the address range sits the **stack for the main thread**, a region of memory that, notably, grows toward lower addresses as it is used — meaning that as more data is pushed onto the stack, the boundary of the stack region extends downward, toward address 0x00000000, rather than upward. Adjacent to the stack lies the **heap**, which by contrast grows in the opposite direction, toward higher addresses, as dynamic memory allocations are made. This opposite-direction growth is a deliberate design choice: it allows the two regions to expand toward one another from a shared pool of available address space without

immediately colliding, maximizing the flexibility available to a program that might need substantially more stack space, more heap space, or some mixture of both, depending on its runtime behavior.

Further along in the address space sits the **program image** itself, which comprises the PE (Portable Executable) header along with the code section, imports, and resources bundled into the executable file. Beyond this lies a region that is allocatable as stack space for additional threads or as additional heap, followed by dedicated areas for **DLLs** (Dynamic Link Libraries) that the process has loaded, each of which occupies its own region of the address space. Finally, near the very top of the user-accessible range, fixed addresses are reserved for structures such as the first Thread Entry Block, the Process Entry Block, and a special page that the kernel shares directly with userland code for certain low-overhead communication purposes.

The critical, deliberately emphasized fact from this memory layout is that the **stack grows toward low addresses**. This single detail has significant practical consequences for reading stack-related assembly code and diagrams: when an analyst observes stack addresses in a debugger or in a diagram, addresses that are numerically lower correspond to more recently pushed, "deeper" positions on the stack, while addresses that are numerically higher correspond to older, earlier stack frames closer to the point where the thread began execution.